

Velora — Copilote Conseiller (POC MCP)

POC du **Learning Lab M2DFS** sur le **Model Context Protocol (MCP)**.

Il comprend :

- un **serveur MCP** (TypeScript) qui expose le catalogue, le stock, les commandes et les politiques de l'e-commerce fictif **Velora** ;
- un **agent IA compatible OpenAI** qui s'en sert pour assister les conseillers, utilisable avec **OpenAI, un modèle local (Ollama / LM Studio) ou tout fournisseur compatible** — configurable uniquement via `.env` .

📄 **Rapport complet** : [docs/rapport/](#) (5 blocs) — version assemblée : [docs/rapport/RAPPORT.md](#) .

🎓 **Atelier + kata** : [docs/atelier/](#) . 🧪 **Preuves d'exécution** : [docs/demo/](#) .

★ Note importante — tester SANS clé payante

L'agent utilise l'**API compatible OpenAI**. Vous pouvez donc le faire tourner :

- avec un **modèle local GRATUIT** (Ollama ou LM Studio) → **aucune clé** ;
- avec **OpenAI** (clé payante) ;
- avec tout autre fournisseur compatible (Groq, Together, OpenRouter...).

Il suffit de modifier les variables `LLM_*` dans `.env` .

De plus, le **serveur MCP ne dépend d'aucun LLM** : il se teste **seul**, sans aucune clé, via `npm test` ou le **MCP Inspector**. Cette même indépendance permet aussi de le brancher sur **Claude Desktop** (cf. [claude_desktop_config.example.json](#)).

Prérequis

- **Node.js 20+** et npm.
- (Optionnel, pour la démo agent gratuite) **Ollama**.

Installation (sans friction)

```
git clone https://github.com/Onitsag/velora-mcp-copilote.git
cd velora-mcp-copilote
npm install
cp .env.example .env      # profil "Ollama local" actif par défaut
npm run db:setup          # crée la base SQLite + données fictives Velora
npm run build              # compile le serveur (dist/)
```

Vérifier que tout marche — sans aucune clé

```
npm test                  # 12 tests (outils MCP + boucle agent simulée)
npm run showcase          # appels réels des outils -> docs/demo/outils-demo.md
npm run inspect           # MCP Inspector sur http://localhost:6274
```

Lancer l'agent (copilote)

Option A — modèle local gratuit (recommandé pour tester)

```
# 1) installer Ollama puis récupérer un modèle qui gère le tool-calling :
ollama pull llama3.1
# (ollama tourne en service ; sinon `ollama serve`)

# 2) .env est déjà réglé sur le profil Ollama. Lancer :
npm run agent -- "Où en est la commande VEL-1003 ?"
npm run demo      # rejoue plusieurs questions -> docs/demo/transcripts/
```

Option B — OpenAI (clé payante)

Dans `.env`, commentez le profil Ollama et décommentez le profil OpenAI (`LLM_MODEL=gpt-4o-mini` recommandé), renseignez `LLM_API_KEY`, puis :

```
npm run agent -- "Avez-vous la robe Lila en taille S ?"
```

⚠ Le modèle choisi doit supporter le **function/tool calling**. Reco locale fiable : `llama3.1` (8B) ; alternatives : `qwen2.5`, `mistral`.

Configuration .env

Variable	Rôle
DATABASE_URL	Base SQLite (laisser file:./dev.db)
LLM_BASE_URL	Endpoint compatible OpenAI (Ollama/OpenAI/LM Studio...)
LLM_API_KEY	Clé API (valeur factice acceptée par Ollama)
LLM_MODEL	Nom du modèle
LLM_TEMPERATURE	Température (défaut 0.2)
AGENT_MAX_STEPS	Nb max d'allers-retours tool-use (défaut 6)

Outils & ressources exposés

Outils : search_products , get_product , check_stock , get_order_status , get_return_policy , create_return_request . **Ressources :** policy://returns , policy://shipping . **Prompt :** conseiller_reply .

Structure du projet

```
src/
  server.ts          # serveur MCP (stdio)
  db.ts              # client Prisma (adapter SQLite)
  tools/             # 1 fichier par outil (Zod + handler)
  resources/         # ressources + politiques (.md)
  prompts/           # prompt conseiller
  agent/             # config, mcpClient, bridge (MCP↔OpenAI), agent, cli
prisma/              # schema + migrations + seed
tests/               # vitest (outils, bridge, boucle agent)
scripts/             # showcase, demo, smoke, build du rapport
docs/                # rapport, atelier (kata), preuves, diagrammes
```

Scripts npm

Script	Effet
<code>npm run db:setup</code>	génère le client + migre + seed
<code>npm run build</code>	compile TypeScript → <code>dist/</code>
<code>npm test</code>	tests (sans clé)
<code>npm run showcase</code>	appels réels des outils → <code>docs/demo/outils-demo.md</code>
<code>npm run inspect</code>	MCP Inspector
<code>npm run agent -- "..."</code>	pose une question au copilote (LLM requis)
<code>npm run demo</code>	transcripts multi-questions (LLM requis)
<code>npm run report:html</code>	assemble le rapport → <code>docs/rapport/RAPPORT.html</code>

Exporter le rapport en PDF

```
npm run report:html
```

Ouvrez `docs/rapport/RAPPORT.html` puis **Imprimer** → **Enregistrer en PDF**. (Le Markdown des `docs/rapport/*.md` se lit aussi directement sur GitHub.)

Sécurité (POC)

Outils en **lecture seule** par défaut ; seule écriture `create_return_request` **restreinte aux commandes livrées** (mention *human-in-the-loop*) ; *system prompt* anti-invention ; schémas d'entrée validés (Zod) ; logs serveur sur stderr.

Stack

TypeScript · `@modelcontextprotocol/sdk` · Prisma 7 + SQLite · SDK `openai` · Zod · Vitest.

Licence

MIT.